

Entity Framework Core 10

What's new and Breaking changes



Entity Framework Core .NET Command-line Tools 10.0.0

SoftUni Team

Technical Trainers



SoftUni



Software University

<https://about.softuni.bg/>

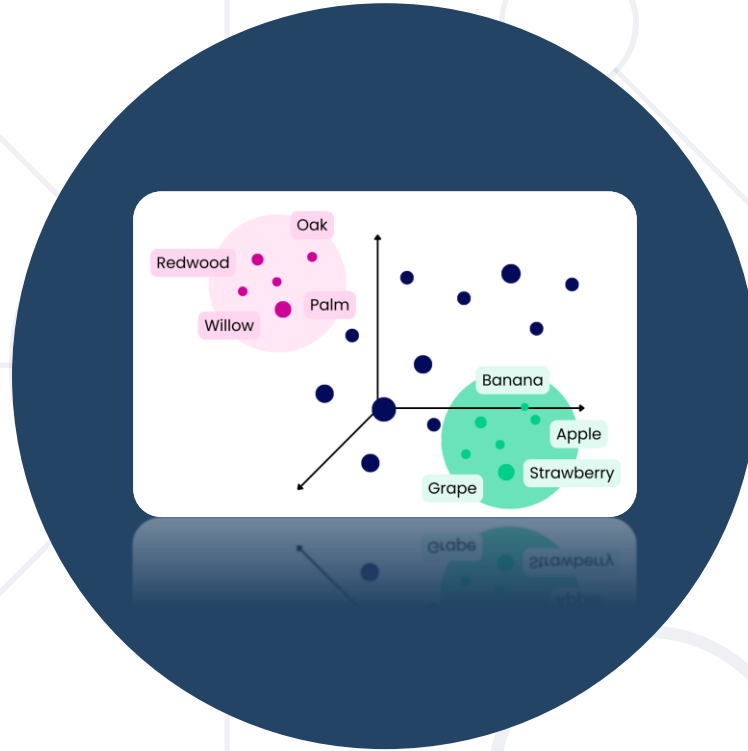
sli.do

#csharp-db

Table of Contents

1. Vector search support
2. Complex Types
3. ExecuteUpdate improvements
4. Named Query Filters
5. LINQ and SQL Translation
6. Breaking Changes





Vector Search Support

Powering AI workloads such as semantic search and RAG

Vector Data Type

- The SQL Server vector data type allows storing *embeddings*
- To use the vector data type, simply add a .NET property of type **SqlVector<float>** to your entity type, specifying the dimensions

⚠ Vector support was introduced in EF Core 10.0 and is only supported with SQL Server 2025 and above.

```
public class Blog
{
    // ...

    [Column(TypeName = "vector(768)")]
    public SqlVector<float> Embedding { get; set; }
}
```

Generate and Store Embeddings

- To generate embeddings, you have to use Embedding Generator
- Microsoft.Extensions.AI provides an interface IEmbeddingGenerator

```
IEmbeddingGenerator<string, Embedding<float>> embeddingGenerator =  
    new OllamaApiClient(new Uri("http://localhost:11434/"), "embeddinggemma");  
  
var embedding = await embeddingGenerator  
    .GenerateVectorAsync("Some text to be vectorized");  
  
context.Blogs.Add(new Blog  
{  
    Name = "Some blog",  
    Embedding = new SqlVector<float>(embedding)  
});  
  
await context.SaveChangesAsync();
```

- Use the `EF.Functions.VectorDistance()` function to perform similarity search

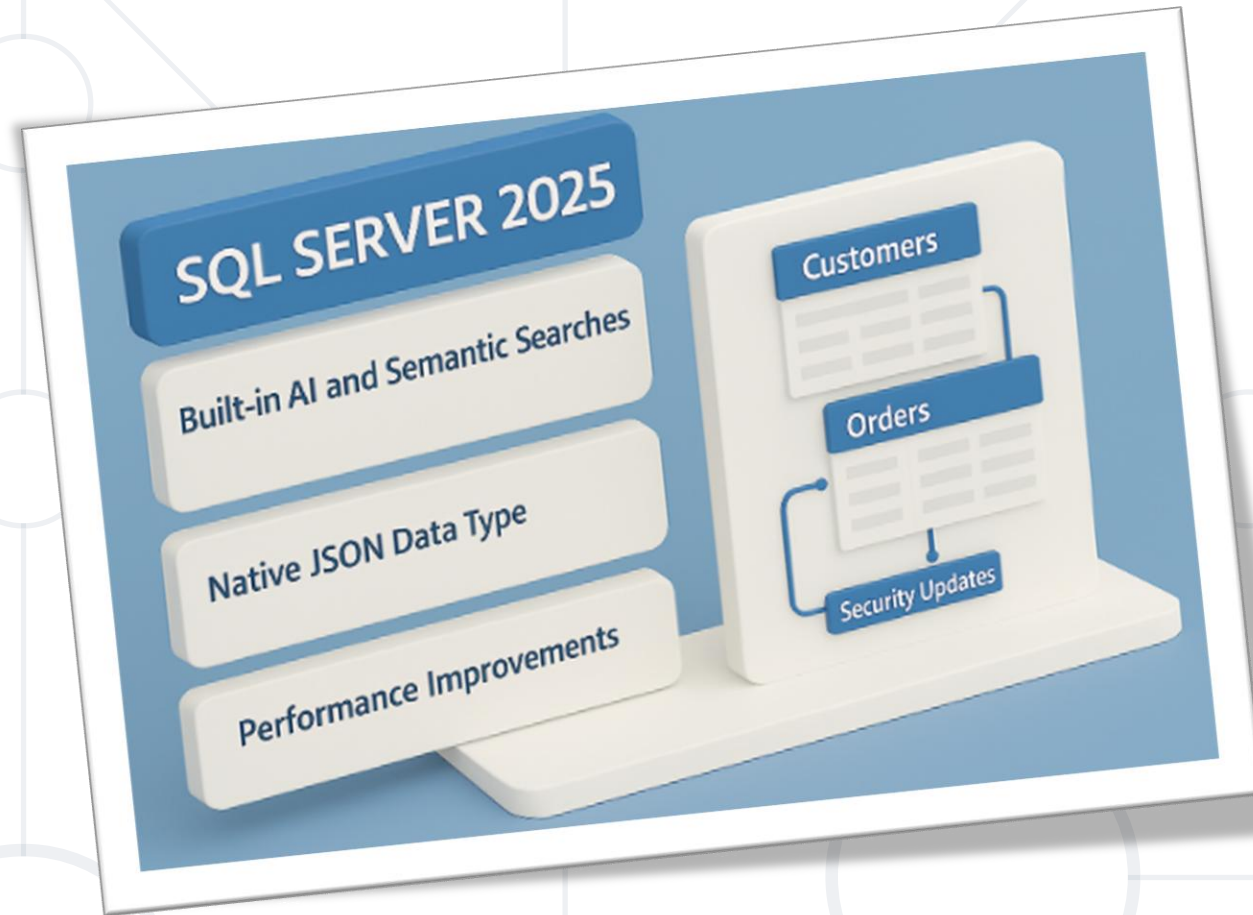
```
var sqlVector = new SqlVector<float>(await embeddingGenerator
    .GenerateVectorAsync("Some user query to be vectorized"));

var topSimilarBlogs = context.Blogs
    .OrderBy(b => EF.Functions
        .VectorDistance("cosine", b.Embedding, sqlVector))
    .Take(3)
    .ToListAsync();
```

Vector Search Tips

- The **vector length** depends on the model (typically, 384–1024 dimensions)
- Commonly used embedding models:
 - embeddinggemma
 - qwen3-embedding
 - all-minilm
- Use the **same embedding model** for both **indexing** and **querying**
- Use **cosine similarity** for most semantic search use cases





Complex Types

JSON and Struct support

- Used to model types which are **contained within** your entity types and have **no identity** of their own
- Can be mapped to **columns in container table** ("table splitting"), or to a single **JSON column**
- Introduce document modeling techniques, which can bring **substantial performance benefits**
- Can make your database modeling much **simpler** and more **natural**

- Map a customer's addresses as **complex types**
- Note the difference with the default, traditional relational behavior of mapping the addresses to a separate table and using a foreign key to represent the customer / address relationship

```
public class Customer
{
    public int Id { get; set; }
    public string Name { get; set; }
    public Address ShippingAddress { get; set; }
    public Address BillingAddress { get; set; }
}
```

```
modelBuilder.Entity<Customer>(b =>
{
    b.ComplexProperty(c => c.ShippingAddress);
    b.ComplexProperty(c => c.BillingAddress);
});
```

```
CREATE TABLE [Customers] (
    [Id] int NOT NULL IDENTITY,
    [Name] nvarchar(max) NOT NULL,
    [BillingAddress_City] nvarchar(max) NOT NULL,
    [BillingAddress_PostalCode] nvarchar(max) NOT NULL,
    [BillingAddress_Street] nvarchar(max) NOT NULL,
    [BillingAddress_StreetNumber] int NOT NULL,
    [ShippingAddress_City] nvarchar(max) NOT NULL,
    [ShippingAddress_PostalCode] nvarchar(max) NOT NULL,
    [ShippingAddress_Street] nvarchar(max) NOT NULL,
    [ShippingAddress_StreetNumber] int NOT NULL,
    CONSTRAINT [PK_Customers] PRIMARY KEY ([Id])
);
```

- EF 10 fully supports the new **JSON data type**, available on SQL Server 2025
- This causes EF to map each Address to a **single JSON column** in the customer table
- Unlike table splitting, JSON mapping **allows collections** within the mapped type
- You can query and update properties **inside JSON documents** just like any other non-JSON property

```
modelBuilder.Entity<Customer>(b =>
{
    b.ComplexProperty(c => c.ShippingAddress, c => c.ToJson());
    b.ComplexProperty(c => c.BillingAddress, c => c.ToJson());
});
```

```
CREATE TABLE [Customers] (
    [Id] int NOT NULL IDENTITY,
    [Name] nvarchar(max) NOT NULL,
    [ShippingAddress] json NOT NULL,
    [BillingAddress] json NOT NULL,
    CONSTRAINT [PK_Customers] PRIMARY KEY ([Id])
);
```

- Complex types also supports mapping **.NET structs** instead of classes

```
public struct Address
{
    public required string Street { get; set; }
    public required string City { get; set; }
    public required string ZipCode { get; set; }
}
```

- This aligns well with complex types **not having an identity** of their own, and only being found within entity types which have an identity
- Collections of structs **aren't currently supported**

Complex and owned entity types

- Owned entity types are entity types, and therefore still operate with reference semantics and an identity behind the scenes
- Trying to assign a customer's billing address to be the same as their shipping address fails with owned entity types, since the same entity type can't be referenced more than once
- since complex types have value semantics, assigning them simply copies their properties over

Owned entity type

```
var customer = await context.Customers.SingleAsync(c  
=> c.Id == someId);  
customer.BillingAddress = customer.ShippingAddress;  
await context.SaveChangesAsync();  
// ERROR
```

Complex type

```
var customer = await context.Customers.SingleAsync(c  
=> c.Id == someId);  
customer.BillingAddress = customer.ShippingAddress;  
await context.SaveChangesAsync();  
// SUCCESS
```





ExecuteUpdate improvements

JSON columns and non-expression lambdas

- **ExecuteUpdate** support for **JSON** requires mapping your types as complex types and does not work when your types are mapped as owned entities

```
public class Blog
{
    public int Id { get; set; }
    public BlogDetails Details { get; set; }
}
```

```
public class BlogDetails
{
    public string Title { get; set; }
    public int Views { get; set; }
}
```

You can now
use **ExecuteUpdate** as usual,
referencing properties
within BlogDetails

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Blog>()
        .ComplexProperty(b => b.Details, bd => bd.ToJson());
}
```

```
await context.Blogs
    .ExecuteUpdateAsync(s =>
        s.SetProperty(b => b.Details.Views,
            b => b.Details.Views + 1));
```

```
UPDATE [b]
SET [Details].modify('$.Views', JSON_VALUE([b].[Details], '$.Views' RETURNING int) + 1)
FROM [Blogs] AS [b]
```


ExecuteUpdateAsync with expression lambda

```
Expression<Func<SetPropertyCalls<Blog>, SetPropertyCalls<Blog>>> setters =  
    s => s.SetProperty(b => b.Views, 8);  
if (nameChanged)  
{  
    var blogParameter = Expression.Parameter(typeof(Blog), "b");  
  
    setters = Expression.Lambda<Func<SetPropertyCalls<Blog>, SetPropertyCalls<Blog>>>(  
        Expression.Call(  
            instance: setters.Body,  
            methodName: nameof(SetPropertyCalls<Blog>.SetProperty),  
            typeArguments: [typeof(string)],  
            arguments:  
            [  
                Expression.Lambda<Func<Blog, string>>(Expression.Property(blogParameter, nameof(Blog.Name)),  
                    blogParameter), Expression.Constant("foo")  
            ]),  
        setters.Parameters);  
}  
await context.Blogs.ExecuteUpdateAsync(setters);
```

- Manually creating **expression trees** is complicated and error-prone, and made this much more difficult than it should have been
- Starting with EF 10, you can now write the following instead

```
await context.Blogs.ExecuteUpdateAsync(s =>
{
    s.SetProperty(b => b.Views, 8);
    if (nameChanged)
    {
        s.SetProperty(b => b.Name, "foo");
    }
});
```



Named Query Filters

- Allows attaching a **filter** to an entity type and having that filter applied whenever a query on that entity type is executed
- Think of them as an additional **LINQ Where** operator that's added whenever the entity type is queried
- Global query filters can be used to filter out **soft-deleted** rows by default, while still allowing you to access them in specific places by disabling the filter for a specific query

```
modelBuilder.Entity<Blog>()  
    .HasQueryFilter(b => !b.IsDeleted);
```

```
var allBlogs = await  
context.Blogs.IgnoreQueryFilters().ToListAsync();
```

Using multiple query filters

- Prior to **EF Core 10** calling **HasQueryFilter** with a simple filter overwrites any previous one

```
modelBuilder.Entity<Blog>().HasQueryFilter(b => !b.IsDeleted);  
// The following overwrites the previous query filter:  
modelBuilder.Entity<Blog>().HasQueryFilter(b => b.TenantId == tenantId);
```

- In order to define **multiple** query filters on the same entity type, they must be **named**. This feature is introduced in EF Core 10

```
modelBuilder.Entity<Blog>()  
    .HasQueryFilter("SoftDeletionFilter", b => !b.IsDeleted)  
    .HasQueryFilter("TenantFilter", b => b.TenantId == tenantId);
```

```
var allBlogs = await context.Blogs  
    .IgnoreQueryFilters(["SoftDeletionFilter"])  
    .ToListAsync();
```



LINQ and SQL Translation

LEFT JOIN (RIGHT JOIN)

Left Join before EF Core 10

```
var query = students
    .GroupJoin(
        departments,
        student => student.DepartmentID,
        department => department.ID,
        (student, departmentList) => new
            { student, subgroup = departmentList })
    .SelectMany(
        joinedSet => joinedSet.subgroup.DefaultIfEmpty(),
        (student, department) => new
        {
            student.student.FirstName,
            student.student.LastName,
            Department = department?.Name ?? string.Empty
        });
```

Left Join EF Core 10 +

```
var query = context.Students
    .LeftJoin(
        context.Departments,
        student => student.DepartmentID,
        department => department.ID,
        (student, department) => new
        {
            student.FirstName,
            student.LastName,
            Department = department.Name ?? "[NONE]"
        });
```

- A notoriously difficult problem with relational databases is queries that involve **parameterized collections**

```
int[] ids = [1, 2, 3];  
var blogs = await context.Blogs.Where(b => ids.Contains(b.Id)).ToListAsync();
```

- EF version up to 8.0 simply **inlined** the collection contents into the SQL as constants

```
SELECT [b].[Id], [b].[Name]  
FROM [Blogs] AS [b]  
WHERE [b].[Id] IN (1, 2, 3)
```

- EF 8.0 leveraged the introduction of extensive **JSON** support and changed the translation of parameterized collections to use **JSON arrays**

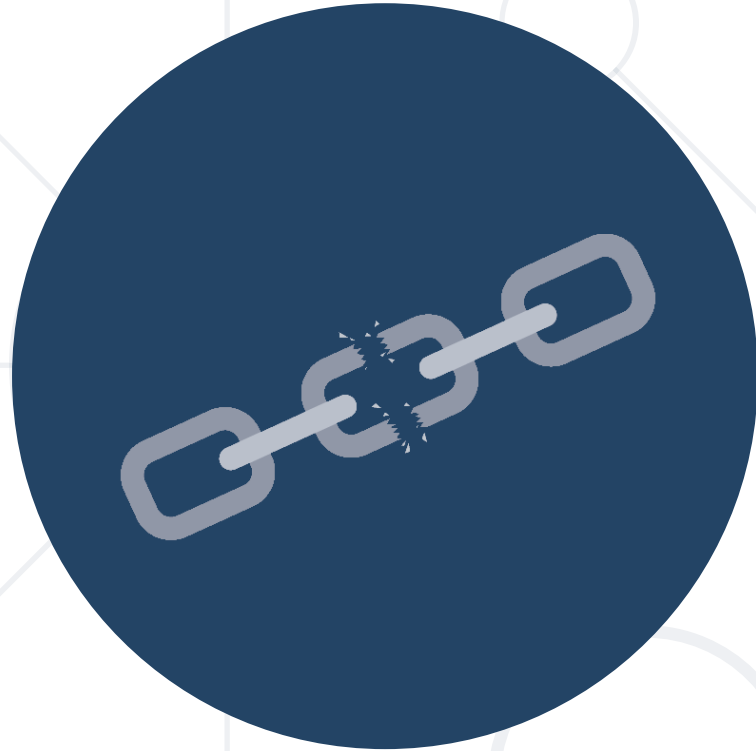
```
@__ids_0='[1,2,3]'  
SELECT [b].[Id], [b].[Name]  
FROM [Blogs] AS [b]  
WHERE [b].[Id] IN (  
    SELECT [i].[value]  
    FROM OPENJSON(@__ids_0) WITH ([value] int '$') AS [i])
```


- EF 10.0 introduces a new default translation mode for parameterized collections, where each value in the collection is translated into its own **scalar parameter**
- Since different cardinalities still cause different SQLs to be generated, EF also **"pads"** parameter list
- Selecting between **multiple parameters**, a single **JSON array parameter** or **multiple inlined constants** can require knowledge about the data in your database

```
SELECT [b].[Id], [b].[Name]
FROM [Blogs] AS [b]
WHERE [b].[Id] IN (@ids1, @ids2, @ids3)
```

```
var blogs = await context.Blogs
    .Where(b => EF.Constant(ids)
    .Contains(b.Id)).ToListAsync();
```

```
protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    => optionsBuilder.UseSqlServer("<CONNECTION STRING>", o =>
        o.UseParameterizedCollectionMode(ParameterTranslationMode.Constant));
```



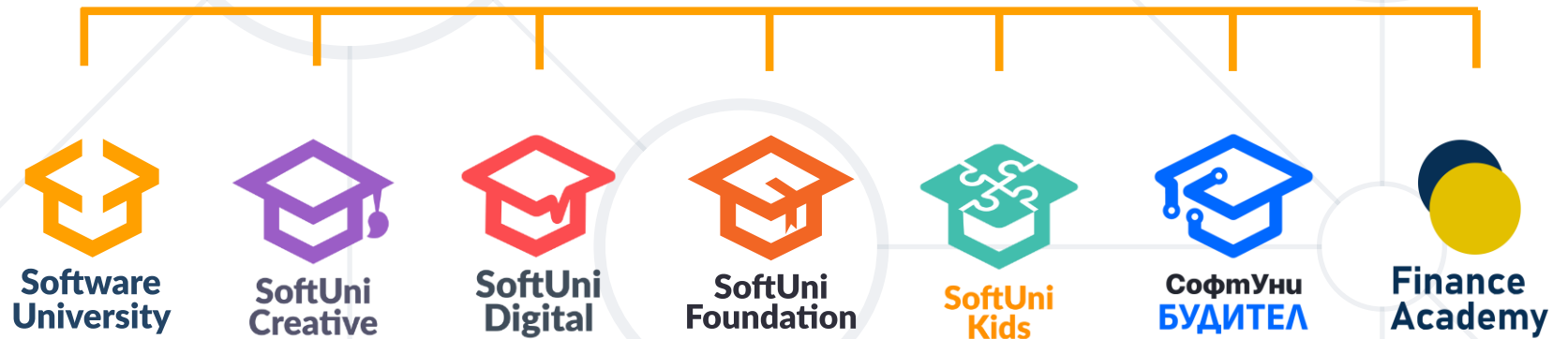
Breaking Changes

- Application Name is now injected into the connection string
- SQL Server json data type used by default on Azure SQL and compatibility level 170
- Parameterized collections now use multiple parameters by default
- ExecuteUpdateAsync now accepts a regular, non-expression lambda
- Complex type column names are now uniquified
- Nested complex type properties use full path in column names
- IDiscriminatorPropertySetConvention signature changed

- **Vectors, Embeddings and Semantic Search**
- Using Complex Types for **Table splitting** and **JSON columns**
- **ExecuteUpdate** Improvements
- Adding multiple **QueryFilters**
- **LEFT JOIN, RIGHT JOIN** and gain control over **Parameterized Collections** SQL translation
- **Breakin Changes** in EF Core 10



Questions?



SoftUni Diamond Partners



THE CROWN IS YOURS



- Software University – High-Quality Education, Profession and Job for Software Developers

- softuni.bg, about.softuni.bg

- Software University Foundation

- softuni.foundation

- Software University @ Facebook

- facebook.com/SoftwareUniversity



- This course (slides, examples, demos, exercises, homework, documents, videos and other assets) is **copyrighted content**
- Unauthorized copy, reproduction or use is illegal
- © SoftUni – <https://about.softuni.bg>
- © Software University – <https://softuni.bg>

